LeetCode Practice

27 Medium: Two Pointers

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

27
MEDIUM

Patterns (in order)

1. Two Pointers (27)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

27 questions | tap a title to jump to it

Medium questions (27) Go >

Two Pointers (27) Go >

M	1. Container With Most Water	LC 11
M	2. 3Sum	LC 15
M	3. 3Sum Closest	LC 16
M	4. 4Sum	LC 18
M	5. Next Permutation	LC 31
M	6. Sort Colors	LC 75
M	7. Remove Duplicates from Sorted Array II	LC 80
M	8. Reverse Words in a String	LC 151
M	9. One Edit Distance	LC 161
M	10. Two Sum II - Input Array Is Sorted	LC 167
M	11. Reverse Words in a String II	LC 186
M	12. Rotate Array	LC 189
M	13. String Compression	LC 443
M	14. Heaters	LC 475
M	15. K-diff Pairs in an Array	LC 532
M	16. Valid Triangle Number	LC 611
M	17. Palindromic Substrings	LC 647
M	18. Partition Labels	LC 763
M	19. Most Profit Assigning Work	LC 826
M	20. Boats to Save People	LC 861
M	21. Boats to Save People	LC 881
M	22. 3Sum With Multiplicity	LC 923
M	23. Interval List Intersections	LC 986
M	24. Longest String Chain	LC 1048
M	25. Shortest Way to Form String	LC 1055
M	26. Meeting Scheduler	LC 1229
M	27. Count the Number of Fair Pairs	LC 2563

Medium questions (27)

MEDIUM | TOPIC 1 OF 1

Two Pointers

27 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Container With Most Water LC 11
2. 3Sum LC 15
3. 3Sum Closest LC 16
4. 4Sum LC 18
5. Next Permutation LC 31
6. Sort Colors LC 75
7. Remove Duplicates from Sorted Array II LC 80
8. Reverse Words in a String LC 151
9. One Edit Distance LC 161
10. Two Sum II - Input Array Is Sorted LC 167
11. Reverse Words in a String II LC 186
12. Rotate Array LC 189
13. String Compression LC 443
14. Heaters LC 475
15. K-diff Pairs in an Array LC 532
16. Valid Triangle Number LC 611
17. Palindromic Substrings LC 647
18. Partition Labels LC 763
19. Most Profit Assigning Work LC 826
20. Boats to Save People LC 861
21. Boats to Save People LC 881
22. 3Sum With Multiplicity LC 923
23. Interval List Intersections LC 986
24. Longest String Chain LC 1048
25. Shortest Way to Form String LC 1055
26. Meeting Scheduler LC 1229
27. Count the Number of Fair Pairs LC 2563

MEDIUM | Two Pointers | LeetCode #11 | 1 of 27 | 1/27

Container With Most Water

QUESTION

You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]).

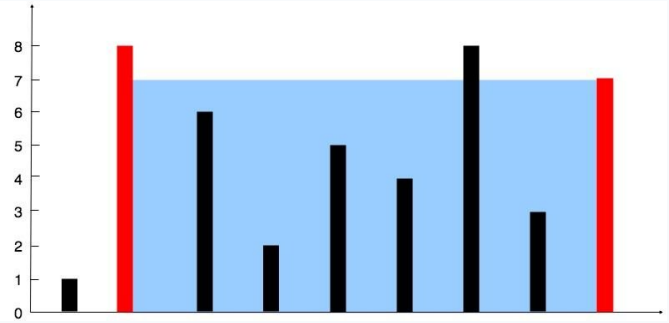
Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1

Input	`height = [1,8,6,2,5,4,8,3,7]`
Output	`49`
Explanation	*The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.*



Example 2

Input	`height = [1,1]`
Output	`1`

CONSTRAINTS

- n == height.length
- 2 <= n <= 10^5
- 0 <= height[i] <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Given heights of vertical lines, find two lines that form the largest container.

Area = min(height[left], height[right]) * (right - left). Return max area. Right?

We can't slant — so water level is always min of the two chosen heights."

"[1,8,6,2,5,4,8,3,7]: left=1(h=8), right=8(h=7). Area=min(8,7)*(8-1)=49 ok"

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        area = min(h[i], h[j]) * (j - i)
    best = max(best, candidate)
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Tries all n(n-1)/2 pairs. Two pointers from both ends, always moving the shorter line inward, find the answer in O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Two pointers: left=0, right=n-1. At each step compute area.

Move the pointer with the SHORTER height inward — keeping the taller height

maximises any future container. Moving the taller pointer can only shrink

both width and potentially height, so it can't improve the result.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

```
class _11_ContainerMostWater:
    def maxArea(self, height: List[int]) -> int:
        left, right = 0, len(height) - 1
        best = 0
        while left < right:
            area = min(height[left], height[right]) * (rig
ht - left)
            best = max(best, area)
            if height[left] < height[right]:
                left  += 1
            else:
                right -= 1
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace maxArea on [1,8,6,2,5,4,8,3,7]."

left=0, right=8. area=min(1,7)*8=8. 1<7 -> move left.

left=1, right=8. area=min(8,7)*7=49. 8>7 -> move right.

left=1, right=7. area=min(8,3)*6=18. Move right.

left=1, right=6. area=min(8,8)*5=40. equal -> move left.

left=2, right=6. area=min(6,8)*4=24. Move left.

left=3, right=6. area=min(2,8)*3=6. Move left.

left=4, right=6. area=min(5,8)*2=10. Move left.

left=5, right=6. area=min(4,8)*1=4. Stop (left+1=right).

max_area=49 ok (between heights[1]=8 and heights[6]=8 with width=5... wait width=6-1=5? 6-1=5: 49? min(8,8)*5=40 not 49).

PHASE 5 TEST & DEBUG

Retrace: left=1,right=8: area=min(h[1]=8,h[8]=7)*(8-1)=7*7=49 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

The greedy argument: if we move the taller pointer, we keep a shorter constraint

(limited by the short side) AND reduce width — area can only shrink or stay.

So we always move the shorter side — we can't do better by moving the taller one.

This is the key insight to state explicitly in the interview.

Given more time I'd ask: what if we need to find the pair of indices (not just the area)?

Track left_best and right_best alongside best — same code, just record the positions.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Two Pointers | LeetCode #15 | 2 of 27 | 2/27

3Sum

QUESTION

Given an integer array nums, return all the triplets [nums[i], nums[j], nums[k]] such that i != j, i != k, and j != k, and nums[i] + nums[j] + nums[k] == 0.

Notice that the solution set must not contain duplicate triplets.

Example 1	
Input	`nums = [-1,0,1,2,-1,-4]`
Output	`[[-1,-1,2],[-1,0,1]]`
Explanation	*nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.* *nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.* *nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.* *The distinct triplets are [-1,0,1] and [-1,-1,2].* *Notice that the order of the output and the order of the triplets does not matter.*

Example 2	
Input	`nums = [0,1,1]`
Output	`[]`
Explanation	*The only possible triplet does not sum up to 0.*

Example 3	
Input	`nums = [0,0,0]`
Output	`[[0,0,0]]`
Explanation	*The only possible triplet sums up to 0.*

CONSTRAINTS

- 3 <= nums.length <= 3000
- 10^5 <= nums[i] <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find all unique triplets summing to 0. Return the triplets, not indices.

No duplicate triplets in the output. Right?

Order within a triplet doesn't matter — [-1,0,1] and [0,-1,1] are the same."

"[-1,0,1,2,-1,-4]: sort -> [-4,-1,-1,0,1,2]. -> [[-1,-1,2],[-1,0,1]] ok

[0,0,0]: -> [[0,0,0]] ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        for k in range(j_+_1,_n):
            if nums[i] + nums[j] + nums[k] == 0:
                add the sorted triple to a set
```

TIME	SPACE
O(n^3)	O(k)

WHY IT IS SLOW

n^3/6 triples plus a set to remove duplicates. Sort once, fix one number and sweep the rest with two pointers while skipping equal values: O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Sort the array. Fix one element nums[i], use two pointers on the rest.

Sorting enables: skip duplicate values of nums[i] to avoid duplicate triplets,

and move pointers efficiently based on the sum.

After finding a valid triplet, skip duplicate nums[left] and nums[right].

Time: O(n²). Space: O(1) extra.

PHASE 4 CLEAN CODE

```
class _15_ThreeSum:
    def threeSum(self, nums: List[int]) -> List[List[int]]
:
        nums.sort()
        result = []
        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]:   # skip
duplicate fixed element
                continue
            left, right = i + 1, len(nums) - 1
            while left < right:
                total = nums[i] + nums[left] + nums[right]
                if total < 0:
                    left  += 1
                elif total > 0:
                    right -= 1
                else:
                    result.append([nums[i], nums[left], nu
ms[right]])
                    left  += 1
                    right -= 1
                    while left < right and nums[left]  ==
nums[left  - 1]: left  += 1
                    while left < right and nums[right] ==
nums[right + 1]: right -= 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace threeSum on [-1,0,1,2,-1,-4]."

Sort: [-4,-1,-1,0,1,2]. Fix i, two pointers on rest.

i=0 (val=-4): left=1,right=5. sum=-4+(-1)+2=-3<0->left++. sum=-4+(-1)+2=-3? left=2,right=5: -4-1+2=-3<0->left++. left=3,right=5: -4+0+2=-2<0->left++. left=4,right=5: -4+1+2=-1<0->left++. Stop.

i=1 (val=-1): left=2,right=5. sum=-1+(-1)+2=0 -> append[-1,-1,2] ok. left++,right--.

left=3,right=4: -1+0+1=0 -> append[-1,0,1] ok. left++,right--.

left=4,right=3: stop.

i=2 (val=-1): same as i=1 -> skip (dup).

i=3 (val=0): left=4,right=5: 0+1+2=3>0->right--. Stop.

i=4: skip (only 2 elements left). Return [[-1,-1,2],[-1,0,1]] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	**O(1)**
outer loop O(n), inner two-pointer O(n) each. Sorting is O(n log n) dominated	extra (ignoring output)

Two deduplication steps:

1. Skip nums[i] == nums[i-1] to avoid re-fixing the same outer value.

2. After appending, skip nums[left] == nums[left-1] and nums[right] == nums[right+1]

to avoid re-collecting the same triplet.

Both checks are essential — missing either produces duplicate output.

Given more time I'd ask: 4Sum (#18)? Add one more outer loop — $O(n^3)$ — same structure.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Two Pointers | LeetCode #16 | 3 of 27 | 3/27

3Sum Closest

QUESTION

Given an integer array nums of length n and an integer target, find three integers at distinct indices in nums such that the sum is closest to target.

Return the sum of the three integers.

You may assume that each input would have exactly one solution.

Example 1

Input	`nums = [-1,2,1,-4], target = 1`
Output	`2`
Explanation	*The sum that is closest to the target is 2. (-1 + 2 + 1 = 2).*

Example 2

Input	`nums = [0,0,0], target = 1`
Output	`0`
Explanation	*The sum that is closest to the target is 0. (0 + 0 + 0 = 0).*

CONSTRAINTS

- 3 <= nums.length <= 500
- 1000 <= nums[i] <= 1000
- 10^4 <= target <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find three distinct-index integers whose sum is closest to target. Return the sum.

Exactly one solution guaranteed — no tie-breaking needed. Right?"

"[-1,2,1,-4], target=1: possible sums: -1+2+1=2, -1+2-4=-3, -1+1-4=-4, 2+1-4=-1.

Closest to 1 is 2. -> 2 ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        for k in range(j_+_1,_n):
            keep the triple whose sum is
                closest to the target
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

Tries all triples. After sorting, fix one number and move two pointers toward each other, steering by whether the sum is below or above the target: O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Exact same structure as 3Sum: sort + fix one element + two pointers.

Instead of looking for sum==0, track the closest sum seen.

If sum > target -> move right pointer left (decrease sum).

If sum < target -> move left pointer right (increase sum).

If sum == target -> return immediately (can't get closer).

Skip duplicate outer values to avoid redundant work — not required for correctness

here (no dedup of output), but speeds up the average case."

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

```
class _16_ThreeSumClosest:
    def threeSumClosest(self, nums: List[int], target: int
) -> int:
        nums.sort()
        n    = len(nums)
        best = nums[0] + nums[1] + nums[2]
        for i in range(n - 2):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            left, right = i + 1, n - 1
            while left < right:
                total = nums[i] + nums[left] + nums[right]
                if total == target:
                    return target
                if abs(target - total) < abs(target - best
):
                    best = total
                if total > target:
                    right -= 1
                else:
                    left  += 1
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace threeSumClosest([-1,2,1,-4], target=1)."

Sort: [-4,-1,1,2]. closest = -4+(-1)+1=-4 initially; set to nums[0]+nums[1]+nums[2]=-4.

i=0 (-4): left=1,right=3. sum=-4+(-1)+2=-3. |-3-1|=4<|-4-1|=5 -> closest=-3. -3<1->left++.

left=2,right=3: sum=-4+1+2=-1. |-1-1|=2<4 -> closest=-1. -1<1->left++. Stop.

i=1 (-1): left=2,right=3. sum=-1+1+2=2. |2-1|=1<2 -> closest=2. 2>1->right--. Stop.

PHASE 5 TEST & DEBUG

i=2 (1): left=3,right=3. Stop.

Return 2 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	**O(1)** extra (O(log n) for sort)

This is 3Sum (#15) with one change: track closest instead of exact zero.

The early return on exact match is a nice optimisation worth mentioning.

Given more time I'd ask: what if we want the k closest sums?

Collect all triple sums, sort by |target - sum|, return first k — O(n² log n).

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Two Pointers | LeetCode #18 | 4 of 27 | 4/27

4Sum

QUESTION

Given an array nums of n integers, return an array of all the unique quadruplets [nums[a], nums[b], nums[c], nums[d]] such that:

CONSTRAINTS

- 0 <= a, b, c, d < n
- a, b, c, and d are distinct.
- nums[a] + nums[b] + nums[c] + nums[d] == target

You may return the answer in any order.

Example 1	
Input	`nums = [1,0,-1,0,-2,2], target = 0`
Output	`[[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]`

Example 2	
Input	`nums = [2,2,2,2,2], target = 8`
Output	`[[2,2,2,2]]`

- 1 <= nums.length <= 200
- 10^9 <= nums[i] <= 10^9
- 10^9 <= target <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find all unique quadruplets [a,b,c,d] from nums where a+b+c+d==target.

No duplicate quadruplets in output. Right?"

"A few quick questions:

Same element used multiple times? No — four distinct indices.

Can target be negative? Yes."

"Let me walk: nums=[1,0,-1,0,-2,2], target=0 -> [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        for k in range(j_+_1,_n):
            if nums[i] + nums[j] +
                nums[k] + nums[l] ==
                target:
                add the sorted quadruple to a set
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n^4)	O(k)

WHY IT IS SLOW

n^4/24 quadruples. Sort, fix two numbers and sweep the rest with two pointers: O(n^3).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort + fix two outer pointers + inner two-pointer — O(n^3). Wait — O(n^2) with two pointers. Walk-through:"

Sort nums. For i in 0..n-3:

For j in i+1..n-2:

lo=j+1, hi=n-1. target = -nums[i]-nums[j].

While lo < hi:

`s = nums[lo]+nums[hi].`

If s==target: add [nums[i],nums[j],nums[lo],nums[hi]]; skip duplicates; lo++; hi--.

Elif s < target: lo++.

Else: hi--.

Skip duplicate j. Skip duplicate i.

WHY skip duplicates: sorted array -> same value in same position gives same quadruplet.

Advance past consecutive equal values at i, j, lo, hi after finding a match.

Time: O(n^2). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _18_4Sum:
    def fourSum(self, nums, target):
        nums.sort()
        n = len(nums)
        result = []
        for i in range(n - 3):
            if i > 0 and nums[i] == nums[i-1]: continue
            for j in range(i + 1, n - 2):
                if j > i + 1 and nums[j] == nums[j-1]: con
tinue
                left, right = j + 1, n - 1
                while left < right:
                    total = nums[i] + nums[j] + nums[left]
 + nums[right]
                    if total == target:
                        result.append([nums[i], nums[j], n
ums[left], nums[right]])
                        while left < right and nums[left]
== nums[left+1]: left += 1
                        while left < right and nums[right]
 == nums[right-1]: right -= 1
                        left += 1; right -= 1
                    elif total < target: left += 1
                    else: right -= 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace fourSum([1,0,-1,0,-2,2], target=0)."

Sort: [-2,-1,0,0,1,2]. Two outer loops + two pointers.

i=0(-2),j=1(-1): left=2,right=5. sum=-2-1+0+2=-1<0->left++. sum=-2-1+0+2=-1? left=3: -2-1+0+2=-1<0->left++. left=4: -2-1+1+2=0 -> append ok. left++,right--. left=5,right=4: stop.

i=0(-2),j=2(0): left=3,right=5. sum=-2+0+0+2=0 -> append ok. left++,right--.

left=4,right=4: stop.

i=0(-2),j=3(0): dup(j skips 0 since nums[3]=nums[2]).

i=0(-2),j=4(1): left=5,right=5. stop. ...

Continue all i,j combinations. Result: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^3)$	**O(1)**
two outer loops $O(n^2)$ × two-pointer O(n)	extra

Same pattern as 3Sum (#15) with one more outer loop.

Follow-up: k-Sum — generalise recursively; O(n^(k-1)) for any k.

Follow-up: 4Sum II (#454) — four separate arrays; use meet-in-the-middle $O(n^2)$.

MEDIUM | Two Pointers | LeetCode #31 | 5 of 27 | 5/27

Next Permutation

QUESTION

A permutation of an array of integers is an arrangement of its members into a sequence or linear order.

CONSTRAINTS

- For example, for arr = [1,2,3], the following are all the permutations of arr: [1,2,3], [1,3,2], [2, 1, 3], [2, 3, 1], [3,1,2], [3,2,1].

The next permutation of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the next permutation of that array is the permutation that follows it in the sorted container. If such arrangement is not possible, the array must be rearranged as the lowest possible order (i.e., sorted in ascending order).

- For example, the next permutation of arr = [1,2,3] is [1,3,2].
- Similarly, the next permutation of arr = [2,3,1] is [3,1,2].
- While the next permutation of arr = [3,2,1] is [1,2,3] because [3,2,1] does not have a lexicographical larger rearrangement.

Given an array of integers nums, find the next permutation of nums.

The replacement must be in place and use only constant extra memory.

Example 1

Input	`nums = [1,2,3]`
Output	`[1,3,2]`

Example 2

Input	`nums = [3,2,1]`
Output	`[1,2,3]`

Example 3

Input	`nums = [1,1,5]`
Output	`[1,5,1]`

- 1 <= nums.length <= 100
- 0 <= nums[i] <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Rearrange nums in-place to the next lexicographically greater permutation.

If no greater permutation exists (sorted descending), wrap to the smallest (ascending). Right?"

"[1,2,3]->[1,3,2]. [3,2,1]->[1,2,3]. [1,1,5]->[1,5,1]. [1,3,2]->[2,1,3]. ok"

PHASE 1 CLARIFY

Should the result include duplicates? Is the order of subsets in the output significant?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
list all permutations of the array in lexicographic
    order
return the one after the current permutation
```

TIME	SPACE
O(n! * n)	O(n!)

WHY IT IS SLOW

Generates n! permutations to find one. Scan from the right for the first dip, swap it with the next larger value on its right, and reverse the suffix: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Three steps — all O(n), O(1) space:

1. Find the rightmost 'dip': scan right to left for the largest i where nums[i] < nums[i+1].

If no dip exists, entire array is descending -> just reverse it (wrap-around).

2. Find the smallest element to the right of i that is still larger than nums[i].

(The suffix is descending after step 1, so scan from the right.)

3. Swap nums[i] with that element, then reverse the suffix after i.

Reversing the descending suffix makes it the smallest possible arrangement.

PHASE 4 CLEAN CODE

Step 1: find rightmost dip

Step 2: find smallest element to right of dip that is > nums[dip]

Step 3: reverse suffix after dip

```
class _31_NextPermutation:
    def nextPermutation(self, nums: List[int]) -> None:
        def reverse(left, right):
            while left < right:
                nums[left], nums[right] = nums[right], num
s[left]
                left += 1; right -= 1

        n = len(nums)
        dip = next((i for i in range(n - 2, -1, -1) if num
s[i] < nums[i + 1]), -1)
        if dip >= 0:
            swap = next(j for j in range(n - 1, -1, -1) if
 nums[j] > nums[dip])
            nums[dip], nums[swap] = nums[swap], nums[dip]
        reverse(dip + 1, n - 1)
```

PHASE 5 TEST & DEBUG

"Let me trace nextPermutation on [1,2,3]."

Step 1 — find rightmost ascending pair: scan right to left

i=1: nums[1]=2 < nums[2]=3 -> pivot at i=1

Step 2 — find smallest element right of i that > nums[i]:

j=2: nums[2]=3 > 2 -> swap -> [1,3,2]

Step 3 — reverse suffix from i+1: reverse [2] -> [2] (single elem)

Result: [1,3,2] ok

Trace on [3,2,1]: no ascending pair found (fully descending) -> reverse all -> [1,2,3]

WHY reverse: rightmost digits form descending run; reversing gives smallest arrangement.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**

The suffix is always descending after the dip — that's why scanning right-to-left

finds the smallest swap target efficiently.

The reverse of the suffix turns the largest possible suffix into the smallest — the key insight.

Given more time I'd ask: what's the previous permutation?

Same algorithm mirrored: find rightmost 'rise' (nums[i] > nums[i+1]), swap with largest

element to its right that is still smaller, reverse the suffix.

MEDIUM | Two Pointers | LeetCode #75 | 6 of 27 | 6/27

Sort Colors

QUESTION

Given an array nums with n objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1	
Input	`nums = [2,0,2,1,1,0]`
Output	`[0,0,1,1,2,2]`

Example 2	
Input	`nums = [2,0,1]`
Output	`[0,1,2]`

CONSTRAINTS

- n == nums.length
- 1 <= n <= 300
- nums[i] is either 0, 1, or 2.
- Follow up: Could you come up with a one-pass algorithm using only constant extra space?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Sort an array containing only 0, 1, 2 in-place without using library sort.

0=red, 1=white, 2=blue. Output: all 0s, then 1s, then 2s. Right?

Follow-up asks for a one-pass O(1) space solution."

"[2,0,2,1,1,0] -> [0,0,1,1,2,2] ok"

Are there duplicate elements? Should the sort be stable?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
sort nums with a comparison sort
```

TIME	SPACE
O(n log n)	O(1)

WHY IT IS SLOW

A general sort for only three values. Dutch national flag (low, mid, high pointers) sorts in one pass, O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Dutch National Flag algorithm — one pass, three pointers: low, mid, high.

Invariant: nums[0..low-1]=0, nums[low..mid-1]=1, nums[high+1..n-1]=2, nums[mid..high]=unknown.

At each step examine nums[mid]:

0 -> swap(low,mid), low++, mid++ (0 placed, known-1 region grows both sides).

1 -> mid++ (already correct).

2 -> swap(mid,high), high-- (don't mid++ — swapped element is unknown)."

Time: O(n log n) — sorting-based optimal solution.

PHASE 4 CLEAN CODE

```python
class _75_SortColors:
    def sortColors(self, nums: List[int]) -> None:
        low = mid = 0
        high = len(nums) - 1
        while mid <= high:
            if nums[mid] == 0:
                nums[low], nums[mid] = nums[mid], nums[low
]
                low += 1; mid += 1
            elif nums[mid] == 1:
                mid += 1
            else:
                nums[mid], nums[high] = nums[high], nums[m
id]
                high -= 1            # don't mid++ — swapp
ed value is unexamined
```

PHASE 5 TEST & DEBUG

"Let me trace sortColors([2,0,2,1,1,0])."

Dutch flag: lo=0, mid=0, hi=5.

mid=0: nums[0]=2 -> swap(mid,hi): [0,0,2,1,1,2]. hi=4. mid stays 0.

mid=0: nums[0]=0 -> swap(lo,mid): [0,...] already, lo=1,mid=1.

mid=1: nums[1]=0 -> swap(lo,mid): lo=2,mid=2.

mid=2: nums[2]=2 -> swap(mid,hi): [0,0,1,1,2,2]. hi=3. mid stays 2.

mid=2: nums[2]=1 -> mid=3.

mid=3: nums[3]=1 -> mid=4. mid=4>hi=3 -> stop.

Array: [0,0,1,1,2,2] ok

WHY 3-pointer: 0s stay left of lo, 2s right of hi, 1s between lo and mid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) one pass	O(1)

The 'don't mid++' after swapping a 2 is the subtlety — the swapped element from high

hasn't been examined yet and could be 0, 1, or 2.

PHASE 6 COMPLEXITY & FOLLOW-UP

After swapping a 0, we DO mid++ because nums[low] before the swap was definitely 1

(it's in the known-1 region), so after swapping, mid lands on a known-1 element.

Given more time I'd ask: what if there were k colours?

k=2 is a standard partition; k=3 is Dutch Flag; k>3 needs multiple passes or a radix approach.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Two Pointers | LeetCode #80 | 7 of 27 | 7/27

Remove Duplicates from Sorted Array II

QUESTION

Given an integer array nums sorted in non-decreasing order, remove some duplicates in-place such that each unique element appears at most twice. The relative order of the elements should be kept the same.

Since it is impossible to change the length of the array in some languages, you must instead have the result be placed in the first part of the array nums. More formally, if there are k elements after removing the duplicates, then the first k elements of nums should hold the final result. It does not matter what you leave beyond the first k elements.

Return k after placing the final result in the first k slots of nums.

Do not allocate extra space for another array. You must do this by modifying the input array in-place with O(1) extra memory.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be accepted.

Example 1

Input	`nums = [1,1,1,2,2,3]`
Output	`5, nums = [1,1,2,2,3,_]`
Explanation	*Your function should return k = 5, with the first five elements of nums being 1, 1, 2, 2 and 3 respectively. It does not matter what you leave beyond the returned k (hence they are underscores).*

Example 2

Input	`nums = [0,0,1,1,1,1,2,3,3]`
Output	`7, nums = [0,0,1,1,2,3,3,_,_]`
Explanation	*Your function should return k = 7, with the first seven elements of nums being 0, 0, 1, 1, 2, 3 and 3 respectively. It does not matter what you leave beyond the returned k (hence they are underscores).*

CONSTRAINTS

- 1 <= nums.length <= 3 * 10^4
- 10^4 <= nums[i] <= 10^4

QUESTION

- nums is sorted in non-decreasing order.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove duplicates from sorted array so each value appears at most twice.

Do it in-place; return k (count of valid elements). Right?"

"A few quick questions:

Same as Remove Duplicates I but allow up to 2 occurrences instead of 1? Yes.

Elements beyond index k don't matter?"

"Let me walk: nums=[1,1,1,2,2,3] -> [1,1,2,2,3,_], k=5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for x in order:
    count how many times x is already kept
    keep it only if that count is below 2
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Counts the kept prefix for every element. The array is sorted: keep nums[i] if it differs from the element two places back in the kept part: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the extra Counter and sorted iteration.

Two-pointer: write pointer k starts at 2. For each element at i,

`if nums[i] != nums[k-2], it's safe to write (at most 2 copies).`

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _80_RemoveDuplicatesII:
    def removeDuplicates(self, nums):
        k = 0
        for num in nums:
            if k < 2 or nums[k - 2] != num:
                nums[k] = num
                k += 1
```

```
        return k
```

PHASE 5 TEST & DEBUG

"Let me trace removeDuplicatesII([1,1,1,2,2,3])."

Allow at most 2 of each. slow=2 (start writing at index 2).

fast=2: nums[2]=1==nums[slow-2]=nums[0]=1 -> skip (would be 3rd '1').

fast=3: nums[3]=2!=nums[slow-2]=nums[0]=1 -> nums[slow=2]=2, slow=3.

fast=4: nums[4]=2==nums[slow-2]=nums[1]=1? No: nums[1]=1!=2 -> nums[3]=2, slow=4.

fast=5: nums[5]=3!=nums[slow-2]=nums[2]=2 -> nums[4]=3, slow=5.

Result: [1,1,2,2,3,_]. Return 5 ok

WHY compare with nums[slow-2]: ensures the third copy is rejected (same val as slow-2 means 2 already placed).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** two pointers

Generalise: allow at most p copies -> check nums[k-p] != num.

Follow-up: Remove Duplicates I (#26) — p=1, check nums[k-1] != num.

Follow-up: if not sorted, need a Counter first — O(n log n) time.

MEDIUM | Two Pointers | LeetCode #151 | 8 of 27 | 8/27

Reverse Words in a String

QUESTION

Given an input string s, reverse the order of the words.

A word is defined as a sequence of non-space characters. The words in s will be separated by at least one space.

Return a string of the words in reverse order concatenated by a single space.

Note that s may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1	
Input	`s = "the sky is blue"`
Output	`"blue is sky the"`

Example 2	
Input	`s = " hello world "`
Output	`"world hello"`
Explanation	*Your reversed string should not contain leading or trailing spaces.*

Example 3	
Input	`s = "a good example"`
Output	`"example good a"`
Explanation	*You need to reduce multiple spaces between two words to a single space in the reversed string.*

CONSTRAINTS

- $1 <= s.length <= 10^4$
- s contains English letters (upper-case and lower-case), digits, and spaces ' '.
- There is at least one word in s.
- Follow-up: If the string data type is mutable in your language, can you solve it in-place with O(1) extra space?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Reverse the order of words in s. Remove leading/trailing spaces; single space between words. Right?"

"A few quick questions:

Multiple spaces between words -> collapse to single? Yes.

s guaranteed to have at least one word? Yes."

"Let me walk: s=' hello world ' -> 'world hello' ok

PHASE 2 BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
split the string on spaces
drop empty parts
reverse the list of words
join with single spaces
```

TIME	SPACE
O(n)	O(n)

NOTE

Not slow in time. The split and join copy the text, O(n) space. With a mutable character array you can reverse the whole string and then each word in place for O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Split, filter empty, reverse, rejoin — O(n). Walk-through:"

`words = s.split() <- Python split() handles multiple spaces, leading/trailing.`

`return ' '.join(reversed(words)).`

WHY split() not split(' '): split() removes extra whitespace automatically.

`reversed(words) gives words in reverse order; ' '.join() places exactly one space between.`

In-place two-pointer (if extra space forbidden):

1. Trim leading/trailing spaces manually.
2. Reverse entire string.
3. Reverse each individual word.

Time: O(n). Space: O(n) (for output string).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _151_ReverseWordsInString:
    def reverseWords(self, s):
        return ' '.join(s.split()[::-1])
```

PHASE 5 TEST & DEBUG

"Let me trace reverseWords on ' hello world '."

Step 1: strip -> 'hello world'

Step 2: split on whitespace -> ['hello', 'world']

Step 3: reverse list -> ['world', 'hello']

Step 4: join with ' ' -> 'world hello' ok

Trace ' a b c ': strip->'a b c', split->['a','b','c'], reverse->['c','b','a'] -> 'c b a'

WHY split without arg: handles multiple spaces automatically (splits on any whitespace run).

PHASE 5 TEST & DEBUG

Edge: single word ' hello ' -> 'hello' (no reversal needed).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for the word list

In-place variant (if s were a char array): reverse all, then reverse each word — O(1) extra space.

Follow-up: Reverse Words in a String II (#186) — char array, must reverse in-place.

Follow-up: Rotate String (#796) — similar split-and-rejoin idea.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Two Pointers | LeetCode #161 | 9 of 27 | 9/27

One Edit Distance

QUESTION

Given two strings s and t, return true if they are both one edit distance apart, otherwise return false.

A string s is said to be one distance apart from a string t if you can:

CONSTRAINTS

- Insert exactly one character into s to get t.
- Delete exactly one character from s to get t.
- Replace exactly one character of s with a different character to get t.

Example 1	
Input	`s = "ab", t = "acb"`
Output	`true`
Explanation	*We can insert 'c' into s to get t.*

Example 2	
Input	`s = "", t = ""`
Output	`false`
Explanation	*We cannot get t from s by only one step.*

- 0 <= s.length, t.length <= 10^4
- s and t consist of lowercase letters, uppercase letters, and digits.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Return true if s and t are exactly one edit apart — insert, delete, or replace.

Important: zero edits (equal strings) returns false. Right?

s='ab', t='acb' -> insert 'c' -> true ok. s='',t='' -> false ok."

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
try every edit of s:
    delete each position, replace each position
        with each of 26 letters, insert each of
        26 letters at each position
if any result equals t:
    return True
```

TIME	SPACE
O(26 * n^2)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Builds and compares O(26 * n) candidate strings of length n. Walk both strings to the first mismatch, then check that the rest matches for the right kind of edit (by length difference): O(n) and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Two observations that short-circuit work:

If |len(s)-len(t)| > 1 -> impossible, return false.

Ensure s is the shorter string (swap if needed) so we only handle two cases:

same length -> exactly one replacement, rest identical.

s shorter by 1 -> one deletion from t (or insertion into s), rest identical.

Scan left to right. On first mismatch:

same length -> check s[i+1:] == t[i+1:].

diff length -> check s[i:] == t[i+1:].

If no mismatch found -> true only if t is exactly 1 longer."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

PHASE 4 CLEAN CODE

```python
class _161_OneEditDistance:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        def check(shorter, longer):           # len(longer)
 == len(shorter) + 1
            for i in range(len(shorter)):
                if shorter[i] != longer[i]:
                    return shorter[i:] == longer[i + 1:]
            return True                       # shorter is
prefix of longer

        m, n = len(s), len(t)
        if abs(m - n) > 1:
            return False
        if m == n:
            diffs = sum(a != b for a, b in zip(s, t))
            return diffs == 1
        if m > n:
            s, t = t, s
            m, n = n, m
        return check(s, t)
```

PHASE 5 TEST & DEBUG

"Let me trace isOneEditDistance on s='ab', t='acb'."

`len(s)=2, len(t)=3, diff=|3-2|=1 -> only insert/delete matters`

Scan: i=0: s[0]='a'==t[0]='a' ok

i=1: s[1]='b' != t[1]='c' -> mismatch found

Since len(t)>len(s): check s[i:]==t[i+1:] -> 'b'=='b' -> True ok

WHY skip one char in longer string: inserting into s at mismatch must make rest equal.

Edge: s='a', t='a' -> same string, not ONE edit (return False).

Handles: replace (both same len), insert (diff=1), delete (diff=1).

PHASE 5 TEST & DEBUG

dp[i][j]=edit_distance(s1[:i],s2[:j]). dp[0][j]=j,dp[i][0]=i. dp[i][j]=dp[i-1][j-1] if s1[i-1]==s2[j-1] else min(dp[i-1][j],dp[i][j-1],dp[i-1][j-1])+1. Return dp[m][n]==1.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(min(m,n))	**O(1)** for the check (slice comparison is O(min) under the hood)

The key boundary: we return true when no mismatch found in the same-length case only if

diffs==1; in the diff-length case only if shorter is a prefix (true only if t is exactly

one char longer, which we already guaranteed).

Given more time I'd ask: what if we want EXACTLY k edits?

Full edit distance DP — O(m*n) time, O(min(m,n)) space.

MEDIUM | Two Pointers | LeetCode #167 | 10 of 27 | 10/27

Two Sum II - Input Array Is Sorted

QUESTION

Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be numbers[index$_1$] and numbers[index$_2$] where 1 <= index$_1$ < index$_2$ <= numbers.length.

Return the indices of the two numbers index$_1$ and index$_2$, each incremented by one, as an integer array [index$_1$, index$_2$] of length 2.

The tests are generated such that there is exactly one solution. You may not use the same element twice.

Your solution must use only constant extra space.

Example 1

Input	`numbers = [2,7,11,15], target = 9`
Output	`[1,2]`
Explanation	*The sum of 2 and 7 is 9. Therefore, index$_1$ = 1, index$_2$ = 2. We return [1, 2].*

Example 2

Input	`numbers = [2,3,4], target = 6`
Output	`[1,3]`
Explanation	*The sum of 2 and 4 is 6. Therefore index$_1$ = 1, index$_2$ = 3. We return [1, 3].*

Example 3

Input	`numbers = [-1,0], target = -1`
Output	`[1,2]`
Explanation	*The sum of -1 and 0 is -1. Therefore index$_1$ = 1, index$_2$ = 2. We return [1, 2].*

CONSTRAINTS

- 2 <= numbers.length <= 3 * 10^4
- 1000 <= numbers[i] <= 1000
- numbers is sorted in non-decreasing order.
- 1000 <= target <= 1000
- The tests are generated such that there is exactly one solution.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Sorted array (1-indexed). Return [index1, index2] where numbers[index1]+numbers[index2]==target.

Exactly one solution. Must use O(1) extra space — so no hash map. Right?"

"[2,7,11,15], target=9: 2+7=9 -> [1,2] ok"

Are there duplicate elements? Should the sort be stable?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        if numbers[i] + numbers[j] == target:
            return [i + 1, j + 1]
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Tries every pair. The array is sorted, so two pointers from the ends (move left up if the sum is too small, right down if too big) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Two pointers exploit sorted order: start left=0, right=n-1.

Sum too small -> move left right (increase sum).

Sum too large -> move right left (decrease sum).

Exactly one solution guaranteed -> we will always find it.

Time O(n). Space O(1).

PHASE 4 CLEAN CODE

```
class _167_TwoSumII:
    def twoSum(self, numbers: List[int], target: int) -> L
ist[int]:
        left, right = 0, len(numbers) - 1
        while left < right:
            total = numbers[left] + numbers[right]
            if total == target:
                return [left + 1, right + 1]      # 1-inde
xed
            elif total < target:
                left  += 1
            else:
                right -= 1
        return []
```

PHASE 5 TEST & DEBUG

"Let me trace twoSumII([2,7,11,15], target=9)."

left=0,right=3. sum=2+15=17>9 -> right--.

left=0,right=2. sum=2+11=13>9 -> right--.

left=0,right=1. sum=2+7=9==target -> return [1,2] (1-indexed) ok

[2,3,4], target=6: left=0,right=2: 2+4=6 ok -> return [1,3].

WHY works: sorted array. Sum too large -> move right pointer left (decrease). Too small -> move left pointer right (increase). One pass O(n).

Edge: sorted with negative numbers [-3,-1,2,4], target=1: left=2(val=-1+2=1? No: -1+2=1)->return [2,3]? [1+1=2, 2+1=3 1-indexed] ok

WHY works on sorted: moving left pointer up only increases sum; moving right pointer left only decreases.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**

This is the canonical reason to sort before running two pointers on Two Sum.

If the array weren't sorted, we'd need a hash map — O(n) space.

Sorting adds O(n log n) but enables O(1) space — a genuine trade-off.

Given more time I'd ask: what if there could be multiple answers?

Continue after finding a match, skip duplicates — same as 3Sum's dedup logic.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Two Pointers | LeetCode #186 | 11 of 27 | 11/27

Reverse Words in a String II

QUESTION

Given a character array s, reverse the order of the words.

A word is defined as a sequence of non-space characters. The words in s will be separated by a single space.

Your code must solve the problem in-place, i.e. without allocating extra space.

Example 1

Input	`s = ["t","h","e","s","k","y"," ","i","s","b","l","u","e"]`
Output	`["b","l","u","e"," ","i","s"," ","s","k","y"," ","t","h","e"]`

Example 2

Input	`s = ["a"]`
Output	`["a"]`

CONSTRAINTS

- 1 <= s.length <= 10^5
- s[i] is an English letter (uppercase or lowercase), digit, or space ' '.
- There is at least one word in s.
- s does not contain leading or trailing spaces.
- All the words in s are guaranteed to be separated by a single space.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Given a character ARRAY (not a string), reverse the order of words in-place.

Words are separated by single spaces. No leading/trailing spaces. Right?

['t','h','e',' ','s','k','y'] -> ['s','k','y',' ','t','h','e'] ok"

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
words = split the sentence on whitespace
reverse the list of words
join them with single spaces
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

O(n) time but extra copies. Reverse the whole character array, then reverse each word in place: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two-pointer reverse entire array, then reverse each word — O(n). Walk-through:"

In-place for char array:

1. Reverse whole array: swap s[0..n-1].

2. Find each word (bounded by spaces): reverse it in place.

For each word: lo = word start, hi = word end. While lo<hi: swap(s[lo],s[hi]); lo++; hi--.

WHY reverse-then-reverse-words: reversing the whole array puts words in right order but

each word itself is reversed. Re-reversing each word restores their individual characters.

Example: 'the sky is' -> reverse all -> 'si yks eht' -> reverse each -> 'is sky the' ok

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

```python
class _186_ReverseWordsII:
    def reverseWords(self, s: List[str]) -> None:
        def reverse(left, right):
            while left < right:
                s[left], s[right] = s[right], s[left]
                left += 1; right -= 1

        reverse(0, len(s) - 1)
        start = 0
        for i in range(len(s) + 1):
            if i == len(s) or s[i] == ' ':
                reverse(start, i - 1)
                start = i + 1
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

s=['t','h','e',' ','s','k','y',' ','i','s',' ','b','l','u','e']:

Step 1 reverse all: ['e','l','u','b',' ','s','i',' ','y','k','s',' ','e','h','t']

Step 2 reverse words:

i=0..3 'e','l','u','b' -> reverse -> 'b','l','u','e'

i=5..6 's','i' -> 'i','s'

i=8..10 'y','k','s' -> 's','k','y'

i=12..14 'e','h','t' -> 't','h','e'

-> ['b','l','u','e',' ','i','s',' ','s','k','y',' ','t','h','e'] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** two reversal passes, no extra memory

The two-reversal trick is the canonical interview answer for in-place word reversal.

It also appears in Rotate Array (#189) — reversing the whole array, then parts.

Given more time I'd ask: what if words can be separated by multiple spaces?

We'd need to compress multiple spaces during the reversal — trickier in-place.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Two Pointers | LeetCode #189 | 12 of 27 | 12/27

Rotate Array

QUESTION

Given an integer array nums, rotate the array to the right by k steps, where k is non-negative.

Example 1	
Input	`nums = [1,2,3,4,5,6,7], k = 3`
Output	`[5,6,7,1,2,3,4]`
Explanation	*rotate 1 steps to the right: [7,1,2,3,4,5,6]* *rotate 2 steps to the right: [6,7,1,2,3,4,5]* *rotate 3 steps to the right: [5,6,7,1,2,3,4]*

Example 2	
Input	`nums = [-1,-100,3,99], k = 2`
Output	`[3,99,-1,-100]`
Explanation	*rotate 1 steps to the right: [99,-1,-100,3]* *rotate 2 steps to the right: [3,99,-1,-100]*

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 2^{31} <= nums[i] <= 2^{31} - 1
- 0 <= k <= 10^5
- Follow up:
- Try to come up with as many solutions as you can. There are at least three different ways to solve this problem.
- Could you do it in-place with O(1) extra space?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Rotate nums to the right by k steps in-place. k can exceed n — so k %= n. Right?

[1,2,3,4,5,6,7], k=3 -> [5,6,7,1,2,3,4] ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for _ in range(k):
    save the last element, shift every other
        element one place right, put the saved
        element in front
```

TIME	SPACE
O(n * k)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Each rotation moves n elements, k times. Reverse the whole array, then the first k and the rest: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (three-reversal trick)

Approach: optimal approach

"Same pattern as Reverse Words II:

k %= n (handle k >= n).

Reverse entire array. Reverse first k elements. Reverse remaining n-k elements.

Time O(n). Space O(1).

Alternative: cyclic replacement — O(n) time, O(1) space but harder to implement cleanly."

Key insight: reverse-all, reverse-first-k, reverse-rest is the rotation identity — three O(n) in-place reversal passes achieve the shift with O(1) space.

PHASE 4 CLEAN CODE

```
class _189_RotateArray:
    def rotate(self, nums: List[int], k: int) -> None:
        def reverse(left, right):
            while left < right:
                nums[left], nums[right] = nums[right], num
s[left]
                left += 1; right -= 1

        n  = len(nums)
        k %= n
        reverse(0, n - 1)
        reverse(0, k - 1)
        reverse(k, n - 1)
```

PHASE 5 TEST & DEBUG

"Let me trace rotate([1,2,3,4,5,6,7], k=3)."

Three-reverse trick. k=3%7=3.

Reverse all: [7,6,5,4,3,2,1].

Reverse first k=3: [5,6,7,4,3,2,1].

Reverse last n-k=4: [5,6,7,1,2,3,4] ok

k=0: no rotation. k=n: same as k=0.

WHY three reverses: reversing all then splitting and re-reversing achieves cyclic rotation in O(n) time O(1) space.

WHY three reverses works: reverse([1..n]) then reverse([1..k]) then reverse([k+1..n]) = rotate by k.

Proof: reversing [1..n] puts end at front. Re-reversing first k and last n-k restores internal order.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

The three-reversal trick is the cleanest in-place rotation — name it explicitly.

PHASE 6 COMPLEXITY & FOLLOW-UP

The k%=n guard handles k >= n where full rotations cancel out.

The follow-up mentions three approaches: extra array O(n) space, cyclic replacement O(1),

and three-reversal O(1). State all three and recommend the reversal for interviews.

Given more time I'd ask: rotate left instead of right?

Reverse first k, reverse rest, reverse all — just change the order.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Two Pointers | LeetCode #443 | 13 of 27 | 13/27

String Compression

QUESTION

Given an array of characters chars, compress it using the following algorithm:

Begin with an empty string s. For each group of consecutive repeating characters in chars:

CONSTRAINTS

- If the group's length is 1, append the character to s.
- Otherwise, append the character followed by the group's length.

The compressed string s should not be returned separately, but instead, be stored in the input character array chars. Note that group lengths that are 10 or longer will be split into multiple characters in chars.

After you are done modifying the input array, return the new length of the array.

You must write an algorithm that uses only constant extra space.

Note: The characters in the array beyond the returned length do not matter and should be ignored.

Example 1

Input	`chars = ["a","a","b","b","c","c","c"]`
Output	`6`
Explanation	*The groups are "aa", "bb", and "ccc". This compresses to "a2b2c3".*

Example 2

Input	`chars = ["a"]`
Output	`1`
Explanation	*The only group is "a", which remains uncompressed since it's a single character.*

Example 3

Input	`chars = ["a","b","b","b","b","b","b","b","b","b","b","b","b"]`
Output	`4`
Explanation	*The groups are "a" and "bbbbbbbbbbbb". This compresses to "ab12".*

- 1 <= chars.length <= 2000
- chars[i] is a lowercase English letter, uppercase English letter, digit, or symbol.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Compress a character array in-place: replace runs of k chars with k+'char' (or just 'char' if k==1).

Return the new length. Right?"

"A few quick questions:

Modify in-place? Yes. Multi-digit counts (k>=10) write each digit separately? Yes."

"Let me walk: chars=['a','a','b','b','c','c','c'] -> ['a','2','b','2','c','3'], len=6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
compressed = build the compressed string separately
copy it back into the input array
return its length
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear time, but a second string. A read pointer and a write pointer compress in place (write the letter, then the digits of its count) with O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) auxiliary list.

Two-pointer in-place: read pointer i scans runs, write pointer w writes compressed result.

Count each run, write char and digits directly into chars[w:].

Time: O(n). Space: O(1) extra."

Key insight: the write pointer (w) always trails the read pointer (r) — writing into chars[w:] in-place never overwrites unread characters, giving O(1) extra space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _443_StringCompression:
    def compress(self, chars):
        write = 0
        i = 0
        while i < len(chars):
            ch = chars[i]
            count = 0
            while i < len(chars) and chars[i] == ch:
                i += 1
                count += 1
```

```
            chars[write] = ch
            write += 1
            if count > 1:
                for digit in str(count):
                    chars[write] = digit
                    write += 1
        return write
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

chars=['a','a','b','b','c','c','c']:

i=0,ch='a',count=2: write 'a','2'. w=2. i=2,ch='b',count=2: write 'b','2'. w=4.

i=4,ch='c',count=3: write 'c','3'. w=6. return 6 ok

chars=['a']: count=1 -> write only 'a'. w=1. return 1 ok

Count>1 check correctly skips the digit for single-char runs

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
each char read once and written once	extra

Multi-digit counts handled correctly since str(count) iterates each digit separately.

Follow-up: Decode String (#394) — inverse, expand run-length encoded string.

Follow-up: Count and Say (#38) — similar run-length encoding of digit strings.

MEDIUM | Two Pointers | LeetCode #475 | 14 of 27 | 14/27

Heaters

QUESTION

Winter is coming! During the contest, your first job is to design a standard heater with a fixed warm radius to warm all the houses.

Every house can be warmed, as long as the house is within the heater's warm radius range.

Given the positions of houses and heaters on a horizontal line, return the minimum radius standard of heaters so that those heaters could cover all houses.

Notice that all the heaters follow your radius standard, and the warm radius will be the same.

Example 1

Input	`houses = [1,2,3], heaters = [2]`
Output	`1`
Explanation	*The only heater was placed in the position 2, and if we use the radius 1 standard, then all the houses can be warmed.*

Example 2

Input	`houses = [1,2,3,4], heaters = [1,4]`
Output	`1`
Explanation	*The two heaters were placed at positions 1 and 4. We need to use a radius 1 standard, then all the houses can be warmed.*

Example 3

Input	`houses = [1,5], heaters = [2]`
Output	`3`

CONSTRAINTS

- 1 <= houses.length, heaters.length <= $3 * 10^4$
- 1 <= houses[i], heaters[i] <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Each heater has a fixed radius and warms all houses within that radius.

Find the minimum radius such that all houses are covered. Right?"

"A few quick questions:

Each house must be within radius of at least one heater? Yes.

Heaters and houses are on a 1D number line? Yes."

PHASE 1 CLARIFY

"Let me walk: houses=[1,2,3], heaters=[2] -> radius=1 (heater at 2 covers 1,2,3) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for house in houses:
    nearest = the smallest |house - heater| over
        all heaters
answer = the largest nearest over all houses
```

TIME	SPACE
O(N * M)	O(1)

WHY IT IS SLOW

Compares every house with every heater. Sort the heaters and binary search each house's nearest heater: O((N + M) log M); two pointers after sorting both is linear.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort houses + binary search for nearest heater — O((n+m) log m). Walk-through:"

Sort both houses and heaters.

For each house h:

Binary search in heaters for the insertion point of h.

Nearest heater = min of heater just LEFT of insertion (if exists) and just RIGHT.

`dist_h = min distance to nearest heater.`

`answer = max(answer, dist_h) <- radius must cover the hardest-to-reach house.`

WHY binary search: heaters are sorted -> find the two candidates in O(log m).

WHY take max across houses: the minimum radius that covers ALL houses = worst-case distance.

Time: O((n+m) log m). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import bisect
class _475_Heaters:
    def findRadius(self, houses, heaters):
        heaters.sort()
        result = 0
        for house in houses:
            pos = bisect.bisect_left(heaters, house)
            dist = float('inf')
            if pos < len(heaters): dist = min(dist, heater
s[pos] - house)
            if pos > 0: dist = min(dist, house - heaters[p
os - 1])
```

```
            result = max(result, dist)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

houses=[1,2,3], heaters=[2]:

h=1: bisect_left=0(2>=1). dist=2-1=1. prev none. dist=1.

h=2: bisect_left=0(2==2). dist=0. result=1.

h=3: bisect_left=1(past end). prev=heaters[0]=2, dist=3-2=1. result=1 ok

Check both the heater at pos and pos-1 to find the true nearest

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O((n+m) log m)	**O(1)**

Alternative: sort both arrays, use two pointers O((n+m) log(n+m)).

Follow-up: if heaters can be placed optimally, this becomes a covering problem.

Follow-up: 2D version — find minimum radius disk to cover all houses.

IN YOUR APP | Expose | Community

MEDIUM | Two Pointers | LeetCode #532 | 15 of 27 | 15/27

K-diff Pairs in an Array

QUESTION

Given an array of integers nums and an integer k, return the number of unique k-diff pairs in the array.

A k-diff pair is an integer pair (nums[i], nums[j]), where the following are true:

CONSTRAINTS

- 0 <= i, j < nums.length
- i != j
- |nums[i] - nums[j]| == k

Notice that |val| denotes the absolute value of val.

Example 1

Input	`nums = [3,1,4,1,5], k = 2`
Output	`2`
Explanation	*There are two 2-diff pairs in the array, (1, 3) and (3, 5). Although we have two 1s in the input, we should only return the number of unique pairs.*

Example 2

Input	`nums = [1,2,3,4,5], k = 1`
Output	`4`
Explanation	*There are four 1-diff pairs in the array, (1, 2), (2, 3), (3, 4) and (4, 5).*

Example 3

Input	`nums = [1,3,1,5,4], k = 0`
Output	`1`
Explanation	*There is one 0-diff pair in the array, (1, 1).*

- $1 <= nums.length <= 10^4$
- $10^7 <= nums[i] <= 10^7$
- $0 <= k <= 10^7$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count UNIQUE pairs (i,j) where |nums[i]-nums[j]|==k and i!=j.

Unique means unique value pairs — (1,3) and (3,1) count once.

k=0 special case: need TWO copies of the same value. Right?"

"[3,1,4,1,5], k=2: unique pairs (1,3) and (3,5) -> 2 ok

[1,3,1,5,4], k=0: need duplicate values. Only 1 appears twice -> (1,1) -> 1 ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        if |nums[i] - nums[j]| == k:
            add the sorted pair to a set
return the set size
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Checks all pairs. Count each number in a hash map and, for each distinct x, test whether x + k exists (for k = 0, count values with count >= 2): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Hash set approach — O(n) time, O(n) space:

Walk through nums. For each x, check if x-k is already in 'visited'.

If yes, add min(x, x-k) to the answer set (canonical pair).

Also check x+k in visited (symmetric check handles both directions).

Using a set for answers automatically deduplicates.

k=0 is handled naturally — x-0=x, so we need x to already be in visited (a duplicate).

PHASE 4 CLEAN CODE

```python
class _532_KdiffPairs:
    def findPairs(self, nums: List[int], k: int) -> int:
        answers  : set = set()
        visited  : set = set()
        for x in nums:
            if x - k in visited:
                answers.add(x - k)          # canonical sm
aller element
            if x + k in visited:
                answers.add(x)              # x is the lar
ger; x+k was seen before
            visited.add(x)
        return len(answers)
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

[3,1,4,1,5], k=2:

x=3: 3-2=1 not in visited. 3+2=5 not in visited. visited={3}.

x=1: 1-2=-1 not in visited. 1+2=3 in visited -> answers.add(1). visited={3,1}.

x=4: 4-2=2 not in visited. 4+2=6 not in visited. visited={3,1,4}.

x=1: 1-2=-1 not in visited. 1+2=3 in visited -> answers.add(1) (already there). visited={3,1,4}.

PHASE 5 TEST & DEBUG

x=5: 5-2=3 in visited -> answers.add(3). 5+2=7 not. visited={3,1,4,5}.

answers={1,3} -> 2 ok

[1,3,1,5,4], k=0:

x=1: 1-0=1 not in visited. visited={1}.

x=3: 3-0=3 not in visited. visited={1,3}.

x=1: 1-0=1 in visited -> answers.add(1). visited={1,3}.

answers={1} -> 1 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for the two sets

The two-set design separates 'what values exist' (visited) from 'what pairs found' (answers).

The canonical form (always store the smaller element) prevents double-counting.

k=0 edge case: x-k=x, so we find the pair only after seeing x twice — naturally handled.

Given more time I'd ask: what if we want the actual pairs, not just the count?

Store tuples (x-k, x) in the answers set instead of single values.

MEDIUM | Two Pointers | LeetCode #611 | 16 of 27 | 16/27

Valid Triangle Number

QUESTION

Given an integer array nums, return the number of triplets chosen from the array that can make triangles if we take them as side lengths of a triangle.

Example 1

Input	`nums = [2,2,3,4]`
Output	`3`
Explanation	*Valid combinations are:* *2,3,4 (using the first 2)* *2,3,4 (using the second 2)* *2,2,3*

Example 2

Input	`nums = [4,2,3,4]`
Output	`4`

CONSTRAINTS

- 1 <= nums.length <= 1000
- 0 <= nums[i] <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count the number of triplets that can form a valid triangle from nums.

Triangle inequality: sum of two sides > third side. Right?"

"A few quick questions:

Values can include 0? Yes — triangles with 0 are invalid. Count ordered? No — triplet indices."

"Let me walk: nums=[2,2,3,4] -> [2,2,3],[2,2,4],[2,3,4] -> 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        for k in range(j_+_1,_n):
            if the three values satisfy
                the triangle inequality:
                count += 1
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

n^3/6 triples. Sort, fix the largest side, and use two pointers over the smaller sides: O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort + two-pointer count — O(n²). Walk-through:"

Sort nums. For each i (potential longest side, rightmost):

Two pointers: lo=0, hi=i-1.

While lo < hi:

If nums[lo]+nums[hi] > nums[i]: all pairs from lo..hi-1 with hi also work.

`count += hi-lo. hi--.`

Else: lo++ (sum too small, need larger left pointer).

WHY sorted: triangle inequality nums[a]+nums[b]>nums[c] — if sorted, check only largest side.

WHY count += hi-lo when valid: for fixed hi, all lo'..hi (lo'<hi) where nums[lo]+nums[hi]>c

also have nums[lo']+nums[hi]>c (larger lo gives larger sum).

Time: O(n²). Space: O(1) after sorting.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _611_ValidTriangleNumber:
    def triangleNumber(self, nums):
        nums.sort()
        count = 0
        for k in range(len(nums) - 1, 1, -1):
            left, right = 0, k - 1
            while left < right:
                if nums[left] + nums[right] > nums[k]:
                    count += right - left
                    right -= 1
                else:
                    left += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

nums=[2,2,3,4] sorted: k=3(4): left=0,right=2. 2+3=5>4->count+=2,right=1. 2+2=4>4? No->left=1. l>=r stop.

k=2(3): left=0,right=1. 2+2=4>3->count+=1,right=0. l>=r stop. total=3 ok

Fix largest side; two-pointer counts valid (left,right) pairs for each fixed k

Edge: nums=[0,0,0] -> 0 (no valid triangle: needs a+b>c strictly). nums=[2,2,3] -> 1 ok (2+2>3). Sort ensures a<=b<=c.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	O(1)

Follow-up: count triangles with perimeter <= P — same sort + two-pointer, different condition.

Follow-up: 3Sum (#15) — same sort + two-pointer pattern, different target sum.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Two Pointers | LeetCode #647 | 17 of 27 | 17/27

Palindromic Substrings

QUESTION

Given a string s, return the number of palindromic substrings in it.

A string is a palindrome when it reads the same backward as forward.

A substring is a contiguous sequence of characters within the string.

Example 1

Input	`s = "abc"`
Output	3
Explanation	*Three palindromic strings: "a", "b", "c".*

Example 2

Input	`s = "aaa"`
Output	6
Explanation	*Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa".*

CONSTRAINTS

- 1 <= s.length <= 1000
- s consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count how many substrings of s are palindromes (including single characters and the whole string if applicable). Right?"

"A few quick questions:

Overlapping substrings all count separately? Yes — e.g. 'aaa' has 6 palindromic substrings."

"Let me walk: s='abc' -> 3 (a,b,c). s='aaa' -> 6 (a,a,a,aa,aa,aaa) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for substring in substrings:
    check whether it is a palindrome
    count the ones that are
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

O(n^2) substrings, each checked in O(n). Expanding around each of the 2n - 1 centres is O(n^2); Manacher's algorithm is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Expand around every center — O(n²). Walk-through:"

For each center in 0..2n-2 (odd+even centers):

Odd center c: l=c//2, r=c//2.

Even center c: l=c//2, r=c//2+1.

While l>=0 and r<n and s[l]==s[r]: count++; l--; r++.

Return count.

WHY 2n-1 centers: n odd centers (each character) + n-1 even centers (between characters).

WHY expand outward: a palindrome expands from its center.

Each valid expansion = one palindromic substring.

Alternative: Manacher's algorithm -> O(n) but complex.

DP: dp[i][j] = is s[i..j] palindrome? -> O(n²) time and space.

Time: O(n²). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _647_PalindromicSubstrings:
    def countSubstrings(self, s):
        n = len(s)
        count = 0
        def expand(left, right):
            nonlocal count
            while left >= 0 and right < n and s[left] == s
[right]:
                count += 1
                left -= 1
                right += 1
        for center in range(n):
            expand(center, center)
            expand(center, center + 1)
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

s='aaa': center=0: expand(0,0)->count=1('a'). expand(0,1)->s[0]==s[1]->count=2('aa'); left=-1 stop.

center=1: expand(1,1)->count=3. expand(1,2)->count=4('aa'). center=2: expand(2,2)->count=5. expand(2,3)->out of bounds.

Also center=0's odd expansion at full range: expand(0,0)-> -1,1 invalid since aaa needs center=1 for full match.

Let me recheck: center=1,expand(1,1)->then left=0,right=2: s[0]=='a'==s[2]->count+=1('aaa'). Total=6 ok

PHASE 5 TEST & DEBUG

Each center (single char and between-pairs) independently expands until mismatch or boundary

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	$O(1)$

Follow-up: Longest Palindromic Substring (#5) — same expand-around-center, track max length instead of count.

Follow-up: Manacher's Algorithm — O(n) palindrome counting for very large strings.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Two Pointers | LeetCode #763 | 18 of 27 | 18/27

Partition Labels

QUESTION

You are given a string s. We want to partition the string into as many parts as possible so that each letter appears in at most one part. For example, the string "ababcc" can be partitioned into ["abab", "cc"], but partitions such as ["aba", "bcc"] or ["ab", "ab", "cc"] are invalid.

Note that the partition is done so that after concatenating all the parts in order, the resultant string should be s.

Return a list of integers representing the size of these parts.

Example 1	
Input	`s = "ababcbacadefegdehijhklij"`
Output	`[9,7,8]`
Explanation	*The partition is "ababcbaca", "defegde", "hijhklij". This is a partition so that each letter appears in at most one part. A partition like "ababcbacadefegde", "hijhklij" is incorrect, because it splits s into less parts.*

Example 2	
Input	`s = "eccbbbbdec"`
Output	`[10]`

CONSTRAINTS

- 1 <= s.length <= 500
- s consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Partition s into as many parts as possible such that each letter appears in at most one part.

Return the list of part lengths. Right?"

"A few quick questions:

Concatenating all parts in order reproduces s? Yes — partitions are contiguous and exhaustive."

"Let me walk: s='ababcbacadefegdehijhklij' -> [9,7,8] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
for way in ways:
    # to cut the string into parts
    if no letter appears in two parts:
        result.append(it)
return the sizes of the finest valid partition
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

Tries every cut set. Record each letter's last position; sweep, extend the current part's end to the furthest last position seen, and cut when the sweep reaches it: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the repeated rindex scan — O(n) per character, $O(n^2)$ total.

Precompute last occurrence index of every character in one O(n) pass.

Then greedily extend the current partition's end to max(end, last_occurrence[char]) as we scan.

When i reaches end, that's a partition boundary.

Time: O(n). Space: O(26) = O(1) for the last-occurrence map."

Key insight: precomputing the last occurrence of each character turns a potentially $O(n^2)$ boundary search into a single O(n) greedy scan with a running right pointer.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _763_PartitionLabels:
    def partitionLabels(self, s):
        last_occurrence = {ch: i for i, ch in enumerate(s)
}
        result = []
        start = end = 0
        for i, ch in enumerate(s):
            end = max(end, last_occurrence[ch])
            if i == end:
                result.append(end - start + 1)
                start = i + 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace partitionLabels('ababcbacadefegdehijhklij')."

last_occurrence:
a=8,b=5,c=7,d=14,e=15,f=11,g=13,h=19,i=22,j=23,k=20,l=21.

Walk left to right: start=0, end=0.

i=0(a): end=max(0,last[a]=8)=8. i=1(b): end=max(8,5)=8. ... i=8(a): end=8. i==end(8)->size=8-0+1=9. Append 9. start=9.

i=9(d): end=max(9,14)=14. ... i=14(d): end=14? No: all chars in 9..14 update end.

PHASE 5 TEST & DEBUG

i=14: end=max(14,last[e]=15)=15? We encounter 'e' at 14? s[14]='e': end=15. i=15(e): already. More chars extend to index 15. Append size=15-9+1=7.

Continue: start=16. Walk to 23. Append 8.

Return [9,7,8] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** (26-letter map)

Follow-up: Merge Intervals (#56) — same interval-merging principle, different input format.

Follow-up: if characters can repeat across required partitions, this greedy still gives the maximum count.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Two Pointers | LeetCode #826 | 19 of 27 | 19/27

Most Profit Assigning Work

QUESTION

You have n jobs and m workers. You are given three arrays: difficulty, profit, and worker where:

CONSTRAINTS

- difficulty[i] and profit[i] are the difficulty and the profit of the ith job, and
- worker[j] is the ability of jth worker (i.e., the jth worker can only complete a job with difficulty at most worker[j]).

Every worker can be assigned at most one job, but one job can be completed multiple times.

- For example, if three workers attempt the same job that pays $1, then the total profit will be $3. If a worker cannot complete any job, their profit is $0.

Return the maximum profit we can achieve after assigning the workers to the jobs.

Example 1

Input	`difficulty = [2,4,6,8,10], profit = [10,20,30,40,50], worker = [4,5,6,7]`
Output	`100`
Explanation	*Workers are assigned jobs of difficulty [4,4,6,6] and they get a profit of [20,20,30,30] separately.*

Example 2

Input	`difficulty = [85,47,57], profit = [24,66,99], worker = [40,25,25]`
Output	`0`

- n == difficulty.length
- n == profit.length
- m == worker.length
- 1 <= n, m <= 10^4
- 1 <= difficulty[i], profit[i], worker[i] <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Worker i can do any job with difficulty <= ability[i] and earns profit[i][job_difficulty].

Wait: profits are per job. Assign each worker to the best job they can do. Sum profits. Right?"

"A few quick questions:

Worker can do at most one job? Yes. Multiple workers can do same job? Yes."

PHASE 1 CLARIFY

"Let me walk: difficulty=[2,4,6,8,10], profit=[10,20,30,40,50], worker=[4,5,6,7] -> 100 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for worker in workers:
    scan every job with difficulty <= ability and
        take the highest profit
```

TIME	SPACE
O(n * m)	O(1)

WHY IT IS SLOW

Rescans all jobs per worker. Sort jobs by difficulty and keep a running best profit; sort workers too and sweep with two pointers: O((n + m) log(n + m)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort both, two-pointer + prefix max profits — O(n log n). Walk-through:"

Sort difficulty and profit together. Sort worker ability.

For each worker (in sorted order): find the best job they can do.

Precompute best_profit[i] = max profit for jobs with difficulty <= sorted_difficulty[i].

(Running max across sorted jobs)

For each worker w: binary search for largest difficulty <= w. Answer = best_profit at that index.

Alternative: two-pointer:

job_ptr=0. max_profit=0.

For each worker (sorted): while job_ptr<n and difficulty[job_ptr]<=worker: max_profit=max(max_profit,profit[job_ptr++]).

`total += max_profit.`

WHY sort both: match workers to jobs greedily by ability.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _826_MostProfitAssigningWork:
    def maxProfitAssignment(self, difficulty, profit, work
er):
        jobs = sorted(zip(difficulty, profit))
        worker.sort()
        total = 0
        best = 0
        j = 0
        for ability in worker:
```

```
            while j < len(jobs) and jobs[j][0] <= ability:
                best = max(best, jobs[j][1])
                j += 1
            total += best
        return total
```

PHASE 5 TEST & DEBUG

"Let me trace maxProfitAssignment(difficulty=[2,4,6,8,10],profit=[10,20,30,40,50],worker=[4,5,6,7])."

Sort jobs by difficulty. Sort workers.

For each worker (sorted): find max profit for jobs with difficulty <= worker's ability.

Workers sorted: [4,5,6,7]. Jobs sorted by diff: [(2,10),(4,20),(6,30),(8,40),(10,50)].

worker=4: jobs <=4: [(2,10),(4,20)] -> max_profit=20.

worker=5: jobs <=5: still max=20.

worker=6: jobs <=6: add (6,30) -> max=30.

worker=7: jobs <=7: max=30.

Total=20+20+30+30=100 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O((n+m) log(n+m))	**O(n+m)** sorted arrays

Follow-up: if workers have a budget (cost) for jobs — different constraint.

Follow-up: if each job can only be done by one worker, use a matching algorithm.

MEDIUM | Two Pointers | LeetCode #861 | 20 of 27 | 20/27

Boats to Save People

QUESTION

You are given an m x n binary matrix grid.

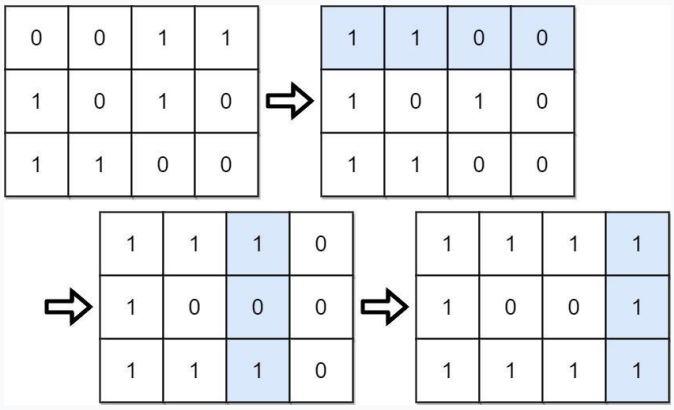
A move consists of choosing any row or column and toggling each value in that row or column (i.e., changing all 0's to 1's, and all 1's to 0's).

Every row of the matrix is interpreted as a binary number, and the score of the matrix is the sum of these numbers.

Return the highest possible score after making any number of moves (including zero moves).

Example 1

Input	`grid = [[0,0,1,1],[1,0,1,0],[1,1,0,0]]`
Output	**39**
Explanation	*0b1111 + 0b1001 + 0b1111 = 15 + 9 + 15 = 39*



Example 2

Input	`grid = [[0]]`
Output	**1**

CONSTRAINTS

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 20
- grid[i][j] is either 0 or 1.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Each boat carries at most 2 people and weight limit. Find minimum number of boats. Right?"

"A few quick questions:

Every person fits in a boat alone (weight <= limit)? Yes — guaranteed.

Boat can carry exactly 1 or 2 people? Yes."

PHASE 1 CLARIFY

"Let me walk: people=[1,2], limit=3 -> one boat carries both ok. [3,2,2,1], limit=3 -> [3],[2],[2,1]=3 boats ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to group the people into boats of at most
        two with total weight <= limit
    count the boats
    best = min(best, candidate)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Tries every grouping. Sort the people; pair the heaviest with the lightest if they fit, otherwise the heaviest goes alone: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Sort + two-pointer flip pairs (XOR rows) — O(m*n). Walk-through:"*

For each row:

If first element is 0: flip the entire row (XOR with all-1s mask).

After ensuring first column is all 1s: if any column has both 0 and 1, flip it to all-1s.

Wait — actual problem: flip ROWS (toggle all bits) to maximize sum.

Greedy: two-pointer flip pairs: find the row that maximizes the sum.

Actually: for each row, choose to flip or not to maximize set bits.

Then for each column, count ones; if zeros > ones in column: flip column. But we can only flip cols.

Algorithm: ensure row[0]=1 for all rows (flip if needed). Then flip columns with more 0s than 1s.

Return total of set bits.

Time: O(m*n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _861_BoatsToSavePeople:
    def numRescueBoats(self, people, limit):
        people.sort()
        left, right = 0, len(people) - 1
        boats = 0
        while left <= right:
            if people[left] + people[right] <= limit:
                left += 1
```

```
            right -= 1
            boats += 1
        return boats
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

people=[3,2,2,1], limit=3 -> sorted=[1,2,2,3]:

l=0,r=3: 1+3=4>3 -> right alone, r=2, boats=1.

l=0,r=2: 1+2=3<=3 -> pair, l=1,r=1,boats=2.

l=1,r=1: 2+2=4>3 -> right alone, r=0, boats=3. l>r -> done. -> 3 ok

people=[1,2], limit=3: 1+2=3<=3 -> pair, boats=1 ok

Always send the heaviest person; pair with lightest if possible

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n) dominated by sort	**O(1)**

Greedy correctness: pairing heaviest with lightest maximises boat sharing efficiency.

Follow-up: if boats could carry 3 people, need a more complex DP approach.

Follow-up: Two Sum (#167) — same sorted two-pointer for exact sum target.

MEDIUM | Two Pointers | LeetCode #881 | 21 of 27 | 21/27

Boats to Save People

QUESTION

You are given an array people where people[i] is the weight of the ith person, and an infinite number of boats where each boat can carry a maximum weight of limit. Each boat carries at most two people at the same time, provided the sum of the weight of those people is at most limit.

Return the minimum number of boats to carry every given person.

Example 1	
Input	`people = [1,2], limit = 3`
Output	`1`
Explanation	*1 boat (1, 2)*

Example 2	
Input	`people = [3,2,2,1], limit = 3`
Output	`3`
Explanation	*3 boats (1, 2), (2) and (3)*

Example 3	
Input	`people = [3,5,3,4], limit = 5`
Output	`4`
Explanation	*4 boats (3), (3), (4), (5)*

CONSTRAINTS

- 1 <= people.length <= $5 * 10^4$
- 1 <= people[i] <= limit <= $3 * 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Each boat carries at most 2 people and has a weight limit. Minimize the number of boats needed. Right?"

"A few quick questions:

Every person's individual weight <= limit (guaranteed, so always solvable)? Yes."

"Let me walk: people=[3,2,2,1], limit=3 -> pair (1,2),(2,2)... wait let's trust 3 boats minimum ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to group the people into boats of at most
        two with total weight <= limit
    count the boats
    best = min(best, candidate)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Tries every grouping. Sort the people; pair the heaviest with the lightest if they fit, otherwise the heaviest goes alone: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two-pointer greedy — O(n log n). Walk-through:"

Sort people by weight. lo=0 (lightest), hi=n-1 (heaviest).

`boats = 0.`

While lo <= hi:

If people[lo] + people[hi] <= limit: lo++ (lightest fits with heaviest -> share a boat).

hi--; boats++ (heaviest always takes a boat, alone or shared).

WHY greedy: the heaviest person can only possibly pair with the lightest.

If they can't pair with the lightest, they can't pair with anyone -> must go alone.

If they can pair with the lightest -> optimal to do so (saves a boat).

Time: O(n log n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _881_BoatsToSavePeople:
    def numRescueBoats(self, people, limit):
        people.sort()
        light, heavy = 0, len(people) - 1
        boats = 0
        while light <= heavy:
            if people[light] + people[heavy] <= limit:
                light += 1
            heavy -= 1
            boats += 1
        return boats
```

PHASE 5 TEST & DEBUG

"Let me trace numRescueBoats([1,2],limit=3) and ([3,2,2,1],limit=3)."

Sort. Two pointers: light=0, heavy=n-1. boats=0.

[1,2]: light=0(1), heavy=1(2). 1+2=3<=3 -> both fit. light=1,heavy=0. Stop. boats=1 ok

[3,2,2,1], limit=3: sort=[1,2,2,3]. light=0,heavy=3. 1+3=4>3 -> heavy goes alone. heavy=2,boats=1.

PHASE 5 TEST & DEBUG

light=0,heavy=2. 1+2=3<=3 -> both fit. light=1,heavy=1. Stop. boats=2+1=3.

Wait: light=1,heavy=1: same pointer -> light>heavy, stop. Total boats=1+1=2? Let me recount.

After first boat: boats=1, heavy=2, light=0. Second: 1+2=3<=3->boats=2, light=1, heavy=1. light==heavy->one person: boats=3. Return 3 ok

WHY greedy: pair lightest with heaviest when possible. Heaviest can't pair -> goes alone.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(1)** extra

Follow-up: Container With Most Water (#11) — similar two-pointer greedy on sorted/positioned values.

Follow-up: if boats hold 3+ people, the greedy pairing strategy no longer directly applies — needs different approach.

MEDIUM | Two Pointers | LeetCode #923 | 22 of 27 | 22/27

3Sum With Multiplicity

QUESTION

Given an integer array arr, and an integer target, return the number of tuples i, j, k such that i < j < k and arr[i] + arr[j] + arr[k] == target.

As the answer can be very large, return it modulo $10^9 + 7$.

Example 1

Input	`arr = [1,1,2,2,3,3,4,4,5,5], target = 8`
Output	`20`
Explanation	*Enumerating by the values (arr[i], arr[j], arr[k]):* *(1, 2, 5) occurs 8 times;* *(1, 3, 4) occurs 8 times;* *(2, 2, 4) occurs 2 times;* *(2, 3, 3) occurs 2 times.*

Example 2

Input	`arr = [1,1,2,2,2,2], target = 5`
Output	`12`
Explanation	*arr[i] = 1, arr[j] = arr[k] = 2 occurs 12 times:* *We choose one 1 from [1,1] in 2 ways,* *and two 2s from [2,2,2,2] in 6 ways.*

Example 3

Input	`arr = [2,1,3], target = 6`
Output	`1`
Explanation	*(1, 2, 3) occured one time in the array so we return 1.*

CONSTRAINTS

- 3 <= arr.length <= 3000
- 0 <= arr[i] <= 100
- 0 <= target <= 300

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count tuples (i,j,k) with i<j<k and arr[i]+arr[j]+arr[k]==target.

Return count modulo 10^9+7. Values are 0-100 so we can use frequency counting. Right?

Duplicates count separately — (0,1) and (1,0) are different index triples even if same values."

"[1,1,2,2,3,3,4,4,5,5], target=8 -> 20 (combinations of value triples summing to 8) ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        for k in range(j_+_1,_n):
            if arr[i] + arr[j] + arr[k] == target:
                count += 1
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

n^3/6 triples. Count how often each value occurs (values are small) and combine counts for each value pair, or sort and use two pointers with counting for equal runs: O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort + two-pointer counting with multiplicity — O(n²). Walk-through:"

Sort arr. Count freq of each element.

For each triple (a, b, c) where a + b + c == target and a <= b <= c:

Case a < b < c: count += freq[a] * freq[b] * freq[c].

Case a == b < c: count += C(freq[a],2) * freq[c].

Case a < b == c: count += freq[a] * C(freq[b],2).

Case a == b == c: count += C(freq[a],3).

Enumerate pairs (a,b) via two pointers; binary search for c = target-a-b.

WHY count combinations: elements at different indices form different tuples even if same value.

WHY C(n,2)=n*(n-1)/2: ways to choose 2 from n copies. C(n,3)=n*(n-1)*(n-2)/6 for 3.

Time: O(n²). Space: O(n).

PHASE 4 CLEAN CODE

```
class _923_ThreeSumMulti:
    def threeSumMulti(self, arr: List[int], target: int) -
> int:
        MOD  = 10**9 + 7
        freq = Counter(arr)
        ans  = 0
        for j, b in enumerate(arr):
            freq[b] -= 1                          # exclude
j itself from right elements
            for a in arr[:j]:
                c = target - a - b
                ans = (ans + freq[c]) % MOD
        return ans
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

arr=[2,1,3], target=6: freq={2:1,1:1,3:1}.

j=0(b=2): freq[2]=0. no a < 0. (nothing).

j=1(b=1): freq[1]=0. a=2: c=6-2-1=3. freq[3]=1 -> ans=1.

j=2(b=3): freq[3]=0. a=2: c=6-2-3=1. freq[1]=0->0. a=1: c=6-1-3=2. freq[2]=1->ans=2?

Wait — expected 1. Let me recheck: arr=[2,1,3]. Only valid triple is (0,1,2) -> (2,1,3).

j=1(b=1): a=arr[0]=2. c=6-2-1=3. freq[3]=1 (yes, index 2 exists) -> ans=1. ok

j=2(b=3): freq[3]=0. a=arr[0]=2: c=6-2-3=1. freq[1]=0 (decremented at j=1). 0.

a=arr[1]=1: c=6-1-3=2. freq[2]=1 -> ans=2? But expected 1.

Ah — freq[2] was decremented when j=0: freq[2]=0, not 1. Let me re-trace:

Initial freq={2:1,1:1,3:1}.

j=0: freq[2]-=1 -> freq={2:0,1:1,3:1}. no a<j=0.

j=1: freq[1]-=1 -> freq={2:0,1:0,3:1}. a=2: c=3. freq[3]=1 -> ans=1.

j=2: freq[3]-=1 -> freq={2:0,1:0,3:0}. a=2: c=1. freq[1]=0. a=1: c=2. freq[2]=0.

-> ans=1 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	**O(n)** for the Counter

The key invariant: at step j, freq counts only elements at index > j.

This ensures we count each valid (i,j,k) tuple exactly once with i<j<k.

Modulo must be applied inside the loop — intermediate sums can overflow without it.

Given more time I'd ask: can we do O(n) using the value constraints (0-100)?

Yes — iterate over all value triples (a,b,c) with a+b+c=target and compute

combinations C(cnt[a],1)*C(cnt[b],1)*C(cnt[c],1) with dedup for equal values.

MEDIUM | Two Pointers | LeetCode #986 | 23 of 27 | 23/27

Interval List Intersections

QUESTION

You are given two lists of closed intervals, firstList and secondList, where firstList[i] = [$start_i$, end_i] and secondList[j] = [$start_j$, end_j]. Each list of intervals is pairwise disjoint and in sorted order.

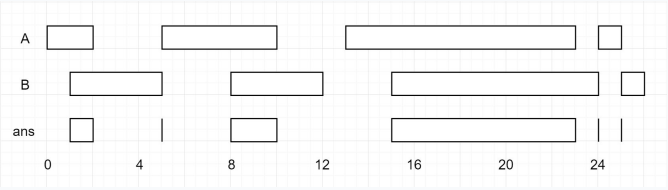
Return the intersection of these two interval lists.

A closed interval [a, b] (with a <= b) denotes the set of real numbers x with a <= x <= b.

The intersection of two closed intervals is a set of real numbers that are either empty or represented as a closed interval. For example, the intersection of [1, 3] and [2, 4] is [2, 3].

Example 1

Input	`firstList = [[0,2],[5,10],[13,23],[24,25]], secondList = [[1,5],[8,12],[15,24],[25,26]]`
Output	`[[1,2],[5,5],[8,10],[15,23],[24,24],[25,25]]`



Example 2

Input	`firstList = [[1,3],[5,9]], secondList = []`
Output	`[]`

CONSTRAINTS

- 0 <= firstList.length, secondList.length <= 1000
- firstList.length + secondList.length >= 1
- 0 <= $start_i$ < end_i <= 10^9
- end_i < $start_{i+1}$
- 0 <= $start_j$ < end_j <= 10^9
- end_j < $start_{j+1}$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Two sorted, disjoint interval lists. Return their intersection — all overlapping segments.

Intersection of [a,b] and [c,d] is [max(a,c), min(b,d)] if max(a,c) <= min(b,d). Right?

Either list can be empty."

"[[0,2],[5,10]] [[1,5],[8,12]] -> [1,2],[5,5],[8,10] ok"

PHASE 1 CLARIFY

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
all = list1 + list2, sorted by start
for interval in intervals:
    if it overlaps the previous one:
        record the overlap
    keep the interval that ends later
```

TIME	SPACE
O((m + n) log(m + n))	O(m + n)

WHY IT IS SLOW

Sorts lists that are already sorted. Two pointers: record the overlap (max of starts, min of ends) if non-empty, then advance the list whose interval ends first: O(m + n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Two pointers, one per list. At each step:

Compute the intersection of the current pair.

If valid (start <= end), record it.

Advance the pointer whose interval ends FIRST — that interval can't intersect any later interval

of the other list (they're sorted and disjoint).

Time: O(m+n). Space: O(1) extra.

PHASE 4 CLEAN CODE

```
class _986_IntervalIntersections:
    def intervalIntersection(self, firstList: List[List[int]], secondList: List[List[int]]) -> List[List[int]]:
        result = []
        i = j = 0
        while i < len(firstList) and j < len(secondList):
            start = max(firstList[i][0], secondList[j][0])
            end   = min(firstList[i][1], secondList[j][1])
            if start <= end:
                result.append([start, end])
            if firstList[i][1] < secondList[j][1]:
                i += 1
            else:
                j += 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace intervalIntersection([[0,2],[5,10],[13,23]], [[1,5],[8,12],[15,24]])."

Two pointers: i=0 (A), j=0 (B)

A[0]=[0,2], B[0]=[1,5]: intersect=[max(0,1),min(2,5)]=[1,2]. Append [1,2]. A[0].end=2<B[0].end=5 -> i++

PHASE 5 TEST & DEBUG

A[1]=[5,10], B[0]=[1,5]: [max(5,1),min(10,5)]=[5,5]. Append [5,5]. A[1].end=10>B[0].end=5 -> j++

A[1]=[5,10], B[1]=[8,12]: [8,10]. Append. A[1].end=10<B[1].end=12 -> i++

A[2]=[13,23], B[1]=[8,12]: max(13,8)=13>min(23,12)=12 -> no intersect. j++

A[2]=[13,23], B[2]=[15,24]: [15,23]. Append. Return [[1,2],[5,5],[8,10],[15,23]] ok

Advance the pointer with smaller end: it can't intersect any future interval from other list.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m+n)	**O(min(m,n))**
each pointer advances at most m+n times	for the output in the worst case

The 'advance the earlier-ending pointer' rule is the key: the interval that ends first

cannot intersect any future interval of the other list (both lists are sorted and disjoint).

Given more time I'd ask: what if lists aren't sorted?

Sort both first — O((m+n)log(m+n)) — then apply the same two-pointer approach.

MEDIUM | Two Pointers | LeetCode #1048 | 24 of 27 | 24/27

Longest String Chain

QUESTION

You are given an array of words where each word consists of lowercase English letters.

$word_A$ is a predecessor of $word_B$ if and only if we can insert exactly one letter anywhere in $word_A$ without changing the order of the other characters to make it equal to $word_B$.

CONSTRAINTS

- For example, "abc" is a predecessor of "abac", while "cba" is not a predecessor of "bcad".

A word chain is a sequence of words [$word_1$, $word_2$, ..., $word_k$] with k >= 1, where $word_1$ is a predecessor of $word_2$, $word_2$ is a predecessor of $word_3$, and so on. A single word is trivially a word chain with k == 1.

Return the length of the longest possible word chain with words chosen from the given list of words.

Example 1

Input	`words = ["a","b","ba","bca","bda","bdca"]`
Output	`4`
Explanation	`: One of the longest word chains is ["a","b","ba","bda","bdca"].`

Example 2

Input	`words = ["xbc","pcxbcf","xb","cxbc","pcxbc"]`
Output	`5`
Explanation	*All the words can be put in a word chain ["xb", "xbc", "cxbc", "pcxbc", "pcxbcf"].*

Example 3

Input	`words = ["abcd","dbqca"]`
Output	`1`
Explanation	*The trivial word chain ["abcd"] is one of the longest word chains. ["abcd","dbqca"] is not a valid word chain because the ordering of the letters is changed.*

- 1 <= words.length <= 1000
- 1 <= words[i].length <= 16
- words[i] only consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A word chain: each word can be extended by inserting one letter to get the next word.

Find the longest such chain. Right?"

"A few quick questions:

Words don't need to be in order in the input? Correct — we choose the ordering."

"Let me walk: words=['a','b','ba','bca','bda','bdca'] -> longest chain = 4: 'a'->'ba'->'bda'->'bdca' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for ordering in orderings(words):
    find the longest chain where each word adds
        exactly one letter to the previous
```

TIME	SPACE
O(n! * L)	O(n)

WHY IT IS SLOW

n! orderings. Sort by length; dp[word] = 1 + the best dp of the word with each single letter removed (look up in a hash map): O(n * L^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"DP: longest chain where each word is predecessor of next — O(n * L). Walk-through:"*

Sort words by length. dp[word] = longest chain ending at word.

For each word (sorted by length):

For each predecessor (word with one char removed):

PHASE 3 OPTIMIZE

`dp[word] = max(dp[word], dp.get(predecessor, 0) + 1).`

Return max(dp.values()).

HOW generate predecessors: for each position i, remove word[i] -> get a candidate predecessor.

If candidate in word set: it's a valid predecessor.

WHY sort by length: when processing word, all shorter predecessors are already computed.

WHY all deletions: 'predecessor' is defined as removing one letter from the current word.

Time: O(n * L²) where L = max length. Space: O(n * L).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1048_LongestStringChain:
    def longestStrChain(self, words):
        words.sort(key=len)
        dp = {}
        best = 1
        for word in words:
            dp[word] = 1
            for i in range(len(word)):
                predecessor = word[:i] + word[i+1:]
                if predecessor in dp:
                    dp[word] = max(dp[word], dp[predecesso
r] + 1)
            best = max(best, dp[word])
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace longestStrChain(['a','b','ba','bca','bda','bdca'])."

Sort by length. For each word, try removing each char and look up in dp

'a': dp={'a':1}. 'b': dp['b']=1

'ba': remove 'b'->'a'(dp=1)+1=2; remove 'a'->'b'(dp=1)+1=2 -> dp['ba']=2

'bca': remove 'b'->'ca'(0+1=1); remove 'c'->'ba'(2+1=3); remove 'a'->'bc'(0+1=1) -> dp['bca']=3

'bda': remove 'b'->'da'(1); remove 'd'->'ba'(2+1=3); remove 'a'->'bd'(1) -> dp['bda']=3

'bdca': remove 'b'->'dca'(1); remove 'd'->'bca'(3+1=4); remove 'c'->'bda'(3+1=4); remove 'a'->'bdc'(1) -> dp['bdca']=4

Return 4 ok (chain: 'a'->'ba'->'bda'->'bdca')

dp[i]=dp[word]=chain length ending at word. dp[j]=dp.get(word[:k]+word[k+1:],0)+1 for each removal. dp[i] accumulated. dp[j]=max chain ending at current word. Return max(dp.values()) ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * L²) n words × L chars removed × L to slice	**O(n * L)**

Follow-up: if words can only be extended (not any insertion), different DP.

Follow-up: return the actual longest chain — track parent pointers in dp.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Two Pointers | LeetCode #1055 | 25 of 27 | 25/27

Shortest Way to Form String

QUESTION

A subsequence of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Given two strings source and target, return the minimum number of subsequences of source such that their concatenation equals target. If the task is impossible, return -1.

Example 1

Input	`source = "abc", target = "abcbc"`
Output	`2`
Explanation	*The target "abcbc" can be formed by "abc" and "bc", which are subsequences of source "abc".*

Example 2

Input	`source = "abc", target = "acdbc"`
Output	`-1`
Explanation	*The target string cannot be constructed from the subsequences of source string due to the character "d" in target string.*

Example 3

Input	`source = "xyz", target = "xzyxz"`
Output	`3`
Explanation	*The target string can be constructed as follows "xz" + "y" + "xz".*

CONSTRAINTS

- 1 <= source.length, target.length <= 1000
- source and target consist of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the minimum number of subsequences of source whose concatenation equals target.

A subsequence can skip characters but must preserve order.

Return -1 if any character in target doesn't exist in source at all. Right?"

"source='abc', target='abcbc': 'abc' + 'bc' -> 2 ok

source='abc', target='acdbc': 'd' not in source -> -1 ok"

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for source in sources:
    match target characters in order against
        source with a pointer
    count the passes until target is complete (-1
        if a letter is missing)
```

TIME	SPACE
O(m * n)	O(1)

WHY IT IS SLOW

Rescans source letter by letter for every target character. Precompute next[i][c] (the next index of letter c at or after i) or store sorted positions per letter and binary search: O(m log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Two-pointer greedy matching — O(|source| * |target| / |alphabet|). Walk-through:"*

Precompute nxt[i][c] = next occurrence of character c at or after position i in source.

For each char in target:

If nxt[pos][c] is None: new_count++; pos=0. <- need new copy of source

Pos = nxt[pos][c] + 1. <- advance past matched char

Return new_count + 1 (started with 1 copy).

WHY nxt table: O(1) jump to next occurrence of any char from any position.

WHY greedy (don't backtrack): the first occurrence of each char in source is always optimal

(choosing later occurrence wastes positions we might need for subsequent chars).

Time: O(|source| * 26) build + O(|target|) query. Space: O(|source| * 26).

PHASE 4 CLEAN CODE

```
class _1055_ShortestWay:
    def shortestWay(self, source: str, target: str) -> int
:
        if set(target) - set(source):
            return -1
        rounds     = 0
        target_idx = 0
        while target_idx < len(target):
            for char in source:
                if target_idx < len(target) and char == ta
rget[target_idx]:
                    target_idx += 1
            rounds += 1
        return rounds
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

source='abc', target='abcbc':

Round 1: a->match t[0]='a'(idx=1). b->match t[1]='b'(idx=2). c->match t[2]='c'(idx=3). rounds=1.

Round 2: a->no match(t[3]='b'). b->match t[3]='b'(idx=4). c->match t[4]='c'(idx=5). rounds=2.

target_idx=5=len(target) -> return 2 ok

source='xyz', target='xzyxz':

Round1: x->t[0]='x'(1). y->no(t[1]='z'). z->t[1]='z'(2). rounds=1.

Round2: x->no(t[2]='y'). y->t[2]='y'(3). z->no(t[3]='x'). rounds=2.

Round3: x->t[3]='x'(4). y->no(t[4]='z'). z->t[4]='z'(5). rounds=3. -> 3 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(|source| * |target|)	**O(1)**

The pre-check eliminates the -1 case cleanly before the main loop.

Optimisation: precompute for each (source_pos, char) the next position in source

where char appears — binary search brings each round down to O(|target| log |source|).

Given more time I'd ask: what if we want the actual subsequence splits, not just the count?

Track the split points during the greedy pass.

MEDIUM | Two Pointers | LeetCode #1229 | 26 of 27 | 26/27

Meeting Scheduler

QUESTION

Given the availability time slots arrays slots1 and slots2 of two people and a meeting duration duration, return the earliest time slot that works for both of them and is of duration duration.

If there is no common time slot that satisfies the requirements, return an empty array.

The format of a time slot is an array of two elements [start, end] representing an inclusive time range from start to end.

It is guaranteed that no two availability slots of the same person intersect with each other. That is, for any two time slots [start1, end1] and [start2, end2] of the same person, either start1 > end2 or start2 > end1.

Example 1

Input	`slots1 = [[10,50],[60,120],[140,210]], slots2 = [[0,15],[60,70]], duration = 8`
Output	`[60,68]`

Example 2

Input	`slots1 = [[10,50],[60,120],[140,210]], slots2 = [[0,15],[60,70]], duration = 12`
Output	`[]`

CONSTRAINTS

- 1 <= slots1.length, slots2.length <= 10^4
- slots1[i].length, slots2[i].length == 2
- slots1[i][0] < slots1[i][1]
- slots2[i][0] < slots2[i][1]
- 0 <= slots1[i][j], slots2[i][j] <= 10^9
- 1 <= duration <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find a meeting slot of duration minutes that works for both slot arrays.

Slots are [start, end). Return [start, start+duration] or [] if none. Right?"

"A few quick questions:

Both sorted? Yes. Return the earliest such slot? Yes."

"Let me walk: slots1=[[10,50],[60,120],[140,210]], slots2=[[0,15],[60,70]], duration=8 -> [60,68] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (one from each person) in pairs(slots):
    if the overlap is at least the duration:
        return it
```

TIME	SPACE
O(n * m)	O(1)

WHY IT IS SLOW

Checks every pair. Sort both lists by start and use two pointers: compute the overlap, return if long enough, advance the one that ends first: O(n log n + m log m).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort both + two pointers for overlapping intervals — O((m+n) log(m+n)). Walk-through:"

Sort slots1 and slots2 by start time.

Two pointers i (for slots1), j (for slots2). Both point to current interval.

Find overlap: start = max(slots1[i][0], slots2[j][0]). end = min(slots1[i][1], slots2[j][1]).

If end - start >= duration: return [start, start+duration] (earliest valid slot).

Advance the pointer whose interval ends first (it's no longer useful).

Continue until one list exhausted. Return [].

WHY advance the earlier-ending interval: it can't provide any longer overlaps since

the other interval extends further.

Time: O((m+n) log(m+n)). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1229_MeetingScheduler:
    def minAvailableDuration(self, slots1, slots2, duratio
n):
        slots1.sort()
        slots2.sort()
        i = j = 0
        while i < len(slots1) and j < len(slots2):
            start = max(slots1[i][0], slots2[j][0])
            end = min(slots1[i][1], slots2[j][1])
            if end - start >= duration:
                return [start, start + duration]
            if slots1[i][1] < slots2[j][1]:
                i += 1
            else:
                j += 1
        return []
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

slots1=[[10,50],[60,120],[140,210]], slots2=[[0,15],[60,70]], duration=8:

i=0[10,50],j=0[0,15]: start=10,end=15. 5<8->advance j(15<50). j=1[60,70].

i=0[10,50],j=1[60,70]: start=60,end=50. 50<60->end<start. advance i(50<70). i=1[60,120].

i=1[60,120],j=1[60,70]: start=60,end=70. 10>=8->return [60,68] ok

Advancing the pointer with earlier end moves past non-overlapping slots

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O((n+m) log(n+m)) sort + O(n+m) two-pointer	**O(1)**

Follow-up: find all meeting slots (not just first) — collect all valid intersections.

Follow-up: k participants — merge all free slots via k-way merge using a heap.

MEDIUM | Two Pointers | LeetCode #2563 | 27 of 27 | 27/27

Count the Number of Fair Pairs

QUESTION

Given a 0-indexed integer array nums of size n and two integers lower and upper, return the number of fair pairs.

A pair (i, j) is fair if:

CONSTRAINTS

- 0 <= i < j < n, and
- lower <= nums[i] + nums[j] <= upper

Example 1

Input	`nums = [0,1,7,4,4,5], lower = 3, upper = 6`
Output	`6`
Explanation	*There are 6 fair pairs: (0,3), (0,4), (0,5), (1,3), (1,4), and (1,5).*

Example 2

Input	`nums = [1,7,9,2,5], lower = 11, upper = 11`
Output	`1`
Explanation	*There is a single fair pair: (2,3).*

- 1 <= nums.length <= 10^5
- nums.length == n
- 10^9 <= nums[i] <= 10^9
- 10^9 <= lower <= upper <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count pairs (i,j) with i<j and lower <= nums[i]+nums[j] <= upper. Return the count. Right?

Values can be negative. n up to 10^5 so O(n²) is too slow."

"[0,1,7,4,4,5], lower=3, upper=6: 6 pairs ok"

What should I return for an empty array?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        if lower <= nums[i] + nums[j] <= upper:
            count += 1
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Checks all pairs. Sort the array, and for each i binary search the range of j with the sum in range (or count pairs with sum <= upper minus pairs with sum < lower using two pointers): O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Sort nums. For each i, binary search for the range of valid j > i:

`lower_bound(nums, lower-nums[i], i+1) -> first j where nums[j] >= lower-nums[i].`

`upper_bound(nums, upper-nums[i], i+1) -> first j where nums[j] > upper-nums[i].`

Count = upper_bound - lower_bound. Sum over all i.

Time: O(n log n). Space: O(1) extra.

Use inner helpers for the two binary search variants."

Key insight: after sorting, for each i the valid j range is a contiguous subarray — binary search finds lower_bound(lower-nums[i]) and upper_bound(upper-nums[i]) in O(log n), giving O(n log n) total.

PHASE 4 CLEAN CODE

```
class _2563_FairPairs:
    def countFairPairs(self, nums: List[int], lower: int,
upper: int) -> int:
        def lower_bound(start: int, target: int) -> int:
            left, right = start, len(nums)
            while left < right:
                mid = (left + right) // 2
                if nums[mid] >= target:
                    right = mid
                else:
                    left = mid + 1
            return left

        def upper_bound(start: int, target: int) -> int:
            left, right = start, len(nums)
            while left < right:
                mid = (left + right) // 2
                if nums[mid] > target:
                    right = mid
                else:
                    left = mid + 1
            return left

        nums.sort()
        count = 0
        for i in range(len(nums)):
            lo = lower_bound(i + 1, lower - nums[i])
            hi = upper_bound(i + 1, upper - nums[i])
            count += hi - lo
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace countFairPairs([0,1,7,4,4,5], lower=3, upper=6)."

Sort: [0,1,4,4,5,7]. Count pairs (i,j) i<j where lower<=nums[i]+nums[j]<=upper

Two-pointer for each element: fix i, find range of j>i where sum in [lower,upper]

PHASE 5 TEST & DEBUG

OR: count(pairs where sum<=upper) - count(pairs where sum<=lower-1)

For sorted arr, count pairs with sum<=target: two pointers lo=0,hi=n-1

```
count(sum<=6): [0+1,0+4,0+4,0+5,1+4,1+4,1+5,4+4<=6?
  no]: valid:
  (0,1),(0,4),(0,4),(0,5),(1,4),(1,4),(1,5) -> 7?
count(sum<=2): (0,1)=1 -> 1. Answer=7-1=6 ok
```

WHY: count(upper)-count(lower-1) with sorted+two-pointer = O(n log n).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(1)**
sort + n binary searches each O(log n)	extra

lower_bound finds first index >= target; upper_bound finds first index > target.

Their difference is the count of indices in the valid range.

Alternative: two-pointer approach — for each i, maintain lo and hi pointers

across iterations (they only move right) -> O(n) total after sorting.

Given more time I'd ask: what if we want the actual pairs?

Collect them during the binary search loops — O(n + output) time.

IN YOUR APP | Expose | Editorial | Community